

Kafka কী জিনিস ? — একদম সহজ ভাষায় (কোনো কোডিং জানা লাগবে না)

এটা লেখা হয়েছে সেইসব ইউনিভার্সিটি শিক্ষার্থীদের জন্য যারা প্রোগ্রামিং জানেন না, কিন্তু "Kafka" জিনিসটা আসলে কী, আর কোনো একটা সফটওয়্যার কোম্পানি (যেমন Sokrio) কেন এটা ব্যবহার করে — সেটা বুঝতে চান। কোনো কোড নেই, শুধু গল্প আর উদাহরণ।

১. একটা রেস্টুরেন্টের গল্প দিয়ে শুরু করি

ধরুন আপনি একটা বড় রেস্টুরেন্টে খেতে গেছেন। আপনি অর্ডার দিলেন ওয়েটারকে। এখন দুইভাবে এটা হতে পারে:

পদ্ধতি ১ (ধীর, কিন্তু সরল): ওয়েটার নিজে রান্নাঘরে গিয়ে দাঁড়িয়ে থাকবে, শেফ আপনার খাবার রান্না করা পর্যন্ত অপেক্ষা করবে, তারপর খাবার নিয়ে আপনার টেবিলে ফিরবে। এই পুরো সময়টা ওয়েটার আর কারও অর্ডার নিতে পারবে না — সে আটকে আছে।

পদ্ধতি ২ (দ্রুত, স্মার্ট): ওয়েটার আপনার অর্ডারটা একটা কাগজে লিখে রান্নাঘরের "অর্ডার বোর্ডে" ঝুলিয়ে দিয়ে সাথে সাথে পরের টেবিলের অর্ডার নিতে চলে যায়। রান্নাঘরে বসে থাকা শেফরা নিজেদের গতিতে বোর্ড থেকে একটার পর একটা অর্ডার তুলে রান্না করে।

Kafka হলো ঠিক এই "অর্ডার বোর্ড"টা — একটা জায়গা যেখানে "কিছু একটা ঘটেছে" (যেমন, নতুন অর্ডার) লিখে রাখা হয়, আর অন্য কেউ (শেফ) নিজের সময়মতো এসে সেটা তুলে নিয়ে কাজ করে। যে অর্ডার লেখে (ওয়েটার) আর যে রান্না করে (শেফ) — এই দুজনকে একে অপরের জন্য অপেক্ষা করতে হয় না। এতে পুরো রেস্টুরেন্ট অনেক দ্রুত চলে।

২. Sokrio-এর আসল সমস্যাটা কী ছিল ?

Sokrio একটা সফটওয়্যার যেটা অনেক কোম্পানিকে তাদের ডিস্ট্রিবিউশন/সাপ্লাই ব্যবসা চালাতে সাহায্য করে। এর মধ্যে দুটো নির্দিষ্ট সমস্যা ছিল:

1. ফিল্ড সেলসম্যানদের **GPS** লোকেশন — মোবাইল অ্যাপ থেকে প্রতি কয়েক সেকেন্ডে একটা করে লোকেশন পয়েন্ট আসে। এত ঘন ঘন ডেটা সাথে সাথে সরাসরি প্রসেস করতে গেলে অ্যাপটাই স্লো হয়ে যায় — যদিও এই একটা পয়েন্ট এক মুহূর্তের মধ্যেই সেভ হওয়া জরুরি না।

2. অর্ডার/ডেলিভারির সামারি রিপোর্ট — আদর্শভাবে এটা প্রায় সাথে সাথেই আপডেট হওয়া উচিত, কিন্তু এখন এটা প্রতি ঘণ্টায় একবার একটা বড় ব্যাচ জব চালিয়ে আপডেট করা হয় (অনেকটা রেস্টুরেন্টে প্রতি ঘণ্টায় একবার সব বিল একসাথে হিসাব করার মতো)।

এই দুই জায়গাতেই সমাধান একই: একটা "অর্ডার বোর্ড" বা বাফার ব্যবহার করা, যাতে যে অংশটা ডেটা তৈরি করছে (যেমন মোবাইল অ্যাপ) আর যে অংশটা সেই ডেটা প্রসেস করছে — তাদের মধ্যে সরাসরি অপেক্ষা করার সম্পর্ক না থাকে।

৩. Kafka-র মূল শব্দগুলো, সহজ উদাহরণ দিয়ে

টেকনিক্যাল শব্দ	সহজ ভাষায় মানে
Topic (টপিক)	রেস্টুরেন্টের একটা আলাদা "অর্ডার বোর্ড", একটা নির্দিষ্ট বিষয়ের জন্য। যেমন একটা বোর্ড শুধু খাবারের অর্ডারের জন্য, আরেকটা শুধু ড্রিংকসের অর্ডারের জন্য।
Producer (প্রোডিউসার)	ওয়েটার — যে বোর্ডে অর্ডার লিখে রাখে। এখানে এটা হতে পারে মোবাইল অ্যাপ বা ওয়েবসাইট, যেটা "এই ঘটনাটা ঘটেছে" বলে জানিয়ে রাখছে।
Consumer (কনজিউমার)	শেফ — যে বোর্ড থেকে অর্ডার তুলে নিয়ে আসল কাজটা (রান্না) করে। এখানে এটা একটা প্রোগ্রাম, যেটা সবসময় চালু থেকে বোর্ড চেক করে আর কাজ করে।
Partition (পার্টিশন)	ধরুন একটা বোর্ড অনেক ব্যস্ত, তাই সেটাকে কয়েকটা ছোট সেকশনে ভাগ করা হলো, যাতে একাধিক শেফ একসাথে আলাদা আলাদা সেকশন থেকে কাজ করতে পারে। একটা নির্দিষ্ট সেকশনের ভেতরে অর্ডারগুলো ঠিক যে ক্রমে লেখা হয়েছে, ঠিক সেই ক্রমেই পড়া হবে — এই গ্যারান্টিটা থাকে।
Partition key (পার্টিশন কী)	কোন অর্ডার কোন সেকশনে যাবে সেটা ঠিক করার নিয়ম। Sokrio-তে নিয়মটা হলো — একই কোম্পানি/ক্লায়েন্টের (tenant) সব অর্ডার সবসময় একই সেকশনে যায়, যাতে সেই একটা কোম্পানির ঘটনাগুলো সবসময় সঠিক ক্রমেই প্রসেস হয়।
Offset (অফসেট)	একটা সেকশনের মধ্যে "আমরা কতদূর পর্যন্ত অর্ডার তুলে ফেলেছি" তার একটা মার্কার/বুকমার্ক। শেফ যদি হঠাৎ বিরতি নিয়ে আবার কাজে ফেরে, তখন এই বুকমার্ক দেখে সে বোঝে কোথা থেকে আবার শুরু করতে হবে — একদম শুরু থেকে না।
Consumer group (কনজিউমার গ্রুপ)	একই কাজ করা শেফদের একটা টিম। এরা নিজেদের মধ্যে সেকশনগুলো ভাগাভাগি করে নেয়, যাতে একই অর্ডার দুইজন শেফ দুইবার না করে ফেলে।

একটা গুরুত্বপূর্ণ পয়েন্ট: এই "বোর্ডে লেখা" জিনিসটা কোনো গ্যারান্টি দেয় না যে কাজটা ১০০% ভাগ্যক্রমে ঠিকঠাক হয়ে যাবে। যদি কোনো শেফ একটা অর্ডার নষ্ট করে ফেলে বা সেটা করতে ব্যর্থ হয়, আর বোর্ড থেকে সেটা মুছেও ফেলে (অফসেট এগিয়ে যায়) — তাহলে ওই একটা অর্ডার হারিয়ে যেতে পারে।

Sokrio-তে ইচ্ছাকৃতভাবে এভাবেই রাখা হয়েছে, কারণ একটা খারাপ অর্ডারের জন্য পুরো বোর্ডটাই আটকে থাকুক — সেটা চায় না।

8. Sokrio-তে বাস্তবে যা ঘটে (একটা উদাহরণ — GPS ট্র্যাকিং)

একজন সেলসম্যানের মোবাইল অ্যাপ থেকে GPS পয়েন্ট পাঠানো হলে কী ঘটে, ধাপে ধাপে দেখা যাক:

1. মোবাইল অ্যাপ একটা লোকেশন পয়েন্ট পাঠায়।
2. সার্ভার সেটা সাথে সাথেই মূল ডেটাবেজে সেভ করে ফেলে (যাতে ডেটা হারিয়ে না যায়), আর একই সাথে সেই তথ্যটা "অর্ডার বোর্ড"-ও (Kafka-তে) টুকে রাখে।
3. একটা আলাদা, সবসময় চালু থাকা প্রোগ্রাম (শেফ) সেই বোর্ড থেকে একে একে GPS পয়েন্ট তুলে নেয় — একটা একটা করে না, বরং প্রতি ২০০টা পয়েন্ট জমা হলে অথবা প্রতি ১ সেকেন্ড পর — যেটা আগে হয়, তখন একবারে একগুচ্ছ পয়েন্ট প্রসেস করে ফেলে। এটা অনেকটা শেফ একবারে একটা করে ডিম না ভেজে, একসাথে ১০টা ডিম ভাজার মতো — সময় বাঁচে।
4. প্রতিটা কোম্পানির (tenant) ডেটা আলাদা আলাদাভাবে সেভ করা হয়, কারণ প্রতিটা কোম্পানির নিজস্ব আলাদা ডেটাবেজ আছে।

এখানে লক্ষণীয় ব্যাপার হলো — ধাপ ২-এর সরাসরি সেভ করাটা এখনও চালু আছে। মানে এখন Kafka মূল ডেটা সেভের একমাত্র রাস্তা না, বরং একটা "পাশাপাশি চলা, পরীক্ষামূলক রাস্তা" — যেটা ভবিষ্যতে হয়তো একমাত্র রাস্তা হয়ে উঠবে, একবার পুরোপুরি ভরসাযোগ্য প্রমাণিত হলে।

৫. এটা ব্যবহার করতে গিয়ে কী কী শেখা হয়েছে (সহজ করে)

Sokrio টিম যখন প্রথমবার এটা চালু করেছিল, তখন কয়েকবার এটা বন্ধ করে আবার নতুন করে ঠিক করে চালু করতে হয়েছিল। কিছু গুরুত্বপূর্ণ শিক্ষা:

- নতুন কিছু চালুর সময়ে পুরনো পদ্ধতিটা সাথে সাথেই বন্ধ করে দেওয়া ঠিক না। রেস্টুরেন্টের উদাহরণেই যদি বলি — নতুন "অর্ডার বোর্ড" পদ্ধতি চালুর প্রথম কয়েকদিন পুরনো "সরাসরি রান্নাঘরে দাঁড়িয়ে থাকা" পদ্ধতিটাও চালু রাখা হয়েছিল, যাতে নতুনটা ব্যর্থ হলেও কোনো অর্ডার হারিয়ে না যায়।
- নতুন পদ্ধতিটা প্রথমে একটা "টেস্ট এলাকায়" রেজাল্ট লিখতো, আসল জায়গায় না — যাতে পুরনো আর নতুন পদ্ধতির ফলাফল পাশাপাশি রেখে মিলিয়ে দেখা যায় দুটো একই রেজাল্ট দিচ্ছে কিনা। মিললেই তবে নতুনটাকে "আসল" জায়গায় লেখার অনুমতি দেওয়া হয়েছে।

- একটা শেফ (কনজিউমার) হঠাৎ কাজ বন্ধ করলেও যেন পুরো রেস্টুরেন্ট না থেমে যায় — তাই একটা কোম্পানির ডেটা প্রসেস করতে সমস্যা হলে, সেটা শুধু লগ করে রাখা হয় আর বাকি কাজ চালিয়ে যাওয়া হয়, পুরো প্রোগ্রামটা ক্র্যাশ করানো হয় না।
 - শেফরা মাঝে মাঝে বিরতি নিয়ে আবার শুরু করে (রিস্টার্ট) — এটা কোনো সমস্যা না, বরং প্রোগ্রামটা দীর্ঘ সময় চালু থাকলে মেমরির সমস্যা এড়াতে ইচ্ছাকৃতভাবেই প্রতি ৬ ঘণ্টা পরপর নিজে থেকে বন্ধ হয়ে আবার চালু হয়।
-

৬. এক লাইনে সারমর্ম

Kafka হলো একটা "মেসেজ রাখার বোর্ড" — যেখানে একটা অংশ (যে ডেটা তৈরি করে) খুব দ্রুত একটা নোট রেখে চলে যায়, আর আরেকটা অংশ (যে ডেটা প্রসেস করে) নিজের সময় ও গতিতে সেই নোট তুলে নিয়ে কাজ করে। এভাবে দুটো অংশ একে অপরের জন্য অপেক্ষা করে বসে থাকে না, পুরো সিস্টেম দ্রুত আর স্থিতিশীল থাকে — ঠিক যেভাবে একটা ব্যস্ত রেস্টুরেন্টে ওয়েটার আর শেফ একসাথে, কিন্তু স্বাধীনভাবে কাজ করে যায়।

৭. ছোট্ট শব্দকোষ (দ্রুত রিভিশনের জন্য)

- **Broker** — যে সার্ভারে "অর্ডার বোর্ড"গুলো আসলে রাখা থাকে।
- **Topic** — একটা নির্দিষ্ট বিষয়ের বোর্ড (যেমন শুধু GPS লোকেশনের বোর্ড)।
- **Partition** — একটা বোর্ডের ভেতরের একটা সেকশন, যাতে একসাথে অনেকে কাজ করতে পারে।
- **Producer** — যে নোট লেখে/অর্ডার দেয়।
- **Consumer** — যে নোট পড়ে আসল কাজটা করে।
- **Offset** — "এতদূর পর্যন্ত পড়া হয়ে গেছে" — তার বুকমার্ক।